

India Runs Hackathon

For a **2-member team + 10 days + winner mindset**, I would split ownership so both members contribute to:

- Python
- AI/ML
- Backend
- Frontend
- Product
- PPT

No one becomes just "frontend" or just "backend".

Day 1 — Dataset Intelligence & Architecture

Member A

Candidate Intelligence Research

- Analyze candidate dataset
- Understand career_history
- Understand skills structure
- Identify hidden signals
- Design Career Trajectory Score

Deliverables:

```
candidate_schema.md  
career_growth_design.md
```

Member B

Job Intelligence Research

- Analyze JD requirements
- Research semantic matching
- Design Top-50 Retrieval Pipeline
- Design ranking architecture

Deliverables:

```
jd_schema.md  
retrieval_design.md
```

Joint

- Final architecture
- Database schema
- API planning

Day 2 — Tier 1 Core Engines

Member A

Candidate Intelligence Engine

Build:

```
candidate_parser.py  
skills_extractor.py  
career_parser.py
```

Feature Ownership:

✓ Candidate Intelligence

Member B

Job Intelligence Engine

Build:

```
jd_parser.py  
requirement_extractor.py
```

Feature Ownership:

✓ JD Intelligence

Day 3 — Semantic Layer

Member A

Candidate Embeddings

Build:

```
candidate_embedding.py
```

Member B

JD Embeddings

Build:

```
semantic_match.py
```

Joint

Merge into:

```
similarity_engine.py
```

Feature Ownership:

☒ Semantic Matching

Day 4 — Ranking Layer

Member A

Build:

```
skill_score.py  
experience_score.py
```

Member B

Build:

```
domain_score.py  
retrieval_engine.py
```

Joint

Build:

```
ranking_engine.py
```

Features:

- ✓ Hybrid Ranking
 - ✓ Top-50 Pipeline
-

Day 5 — Tier 2 Features

Member A

Evidence Layer

Build:

```
evidence_engine.py  
missing_skills.py
```

Features:

- ✓ Evidence Ranking
 - ✓ Missing Skills
-

Member B

Dashboard

Build:

```
Dashboard  
Ranking Table
```

Features:

- ✓ Recruiter Dashboard
-

Day 6 — Tier 3 Features

Member A

Career Intelligence

Build:

```
career_trajectory.py
```

Feature:

✓ Career Trajectory Score

Hidden Gem Detector

Build:

```
hidden_gem_detector.py
```

Feature:

✓ Hidden Gem Detection

Member B

Comparison System

Build:

```
comparison_engine.py  
comparison_page.tsx
```

Feature:

✓ Candidate Comparison

Day 7 — Tier 3 Features Continued

Member A

Potential Engine

Build:

```
potential_score.py
```

Feature:

☒ Potential Score

Member B

Risk Engine

Build:

```
risk_analysis.py
```

Feature:

☒ Risk Analysis

Joint

Merge scores into ranking engine.

Day 8 — Tier 3 & Tier 4

Member A

Explainability

Build:

```
explanation_engine.py
```

Features:

- ✓ Explainable Ranking
 - ✓ Why Candidate A > Candidate B
-

Member B

Recruiter Recommendation

Build:

```
recommendation_engine.py
```

Feature:

- ✓ Recruiter Recommendation Engine
-

Day 9 — Tier 4

Member A

Anonymous Mode Backend

Build:

```
anonymizer.py
```

Feature:

- ✓ Anonymous Hiring Mode
-

Member B

Visualization Layer

Build:

```
funnel_visualization.tsx
```

Feature:

✓ Hiring Funnel Visualization

Joint

Integrate:

- Recommendation
 - Anonymous Mode
 - Funnel
-

Day 10 — Polish & Submission

Member A

- CSV Generator
- Performance Optimization
- Model Tuning

Features:

✓ CSV Output

Member B

- UI Polish
 - Demo Flow
 - Animations
-

Joint

PPT

Slides:

1. Problem
2. Dataset
3. Architecture
4. Pipeline
5. Ranking Formula
6. Hidden Gem Detector
7. Potential Score
8. Career Trajectory
9. Results
10. Demo
11. Impact

Demo

Flow:

```
JD Input
↓
AI Understanding
↓
Top 50 Retrieval
↓
Ranking
↓
Candidate Comparison
↓
Hidden Gem
↓
Potential Analysis
↓
Recruiter Recommendation
↓
```

Anonymous Mode

↓

Export

Final Feature Ownership

Feature	A	B
JD Intelligence		✓
Candidate Intelligence	✓	
Semantic Matching	✓	✓
Hybrid Ranking	✓	✓
Top-50 Pipeline		✓
Evidence Ranking	✓	
Missing Skills	✓	
Candidate Dashboard		✓
Candidate Detail View		✓
Career Trajectory Score	✓	
Hidden Gem Detector	✓	
Candidate Comparison		✓
Potential Score	✓	
Risk Analysis		✓
Explainable Ranking	✓	
Why A > B	✓	
Recruiter Recommendation		✓
Anonymous Mode	✓	
Hiring Funnel Visualization		✓
CSV Export	✓	
PPT	✓	✓
Demo	✓	✓

This split is close to **50–50 effort**, gives both members meaningful AI and development work, and covers all Tier 1–4 features without one person carrying the entire project.

EXPLANATION OF EVERY FEATURE

Send this to your teammate. It explains **what each feature does, why it exists, how it works, and its impact on winning.**

Tier 1 Features (Core System)

These solve the hackathon's basic requirement.

1. JD Intelligence Engine

What it does

Reads the Job Description and understands:

- Required skills
- Experience level
- Role type
- Domain
- Must-have requirements

Example

JD:

```
Looking for Backend Engineer
```

```
Skills:  
Python  
FastAPI
```

PostgreSQL

Experience:
3+ years

Output:

```
{  
  "role": "Backend Engineer",  
  "skills": ["Python", "FastAPI", "PostgreSQL"],  
  "experience": 3  
}
```

Why important

Without understanding the job, ranking is useless.

2. Candidate Intelligence Engine

What it does

Converts candidate data into structured format.

Extracts:

- Skills
- Experience
- Projects
- Career history
- Education

Example

Candidate profile:

Software Engineer
Worked with Python and Django

Built APIs

Output:

```
{  
  "skills":["Python","Django","API"],  
  "experience":4  
}
```

Why important

Makes all candidates comparable.

3. Semantic Matching

What it does

Matches meaning instead of keywords.

Example

JD:

Machine Learning Engineer

Candidate:

AI Engineer
Deep Learning
NLP

Traditional ATS:

No exact keyword

Your system:

Strong Match

Why important

Core problem statement.

4. Hybrid Ranking Engine

What it does

Combines multiple scores.

Example:

40% Semantic Match
25% Skill Match
15% Experience
10% Domain Fit
10% Growth

Final:

Candidate Score = 87

Why important

Main ranking logic.

5. Explainable Ranking

What it does

Explains every ranking.

Instead of:

87%

Shows:

Matched:
Python
FastAPI
PostgreSQL

Experience:
4 years

Projects:
Backend APIs

Why important

Judges specifically mentioned trust and explainability.

6. Top-50 Retrieval Pipeline

What it does

Filters candidates in stages.

5000 Candidates
↓
500 Relevant
↓
50 Strong Matches
↓
10 Recommended

Why important

Production-style hiring workflow.

7. CSV Export

What it does

Exports final ranked shortlist.

Why important

Mandatory hackathon requirement.

Tier 2 Features

These make it look like a real product.

8. Candidate Comparison

What it does

Compare two candidates side-by-side.

Example:

Metric	A	B
Skills	9	8
Experience	8	9
Growth	10	6

Why important

Recruiters compare candidates constantly.

9. Evidence-Based Ranking

What it does

Shows proof behind scores.

Example:

Matched Python

Evidence:

Worked 4 years in Python

Why important

Makes rankings trustworthy.

10. Missing Skills Analysis

What it does

Shows gaps.

Example:

Matched:

Python

FastAPI

Missing:

AWS

Docker

Why important

Helps recruiters make decisions.

11. Recruiter Dashboard

What it does

Main screen.

Contains:

- JD Upload
- Rankings
- Filters
- Insights

Why important

Judges see this first.

12. Candidate Detail View

What it does

Detailed profile page.

Contains:

- Skills
- Growth
- Risk
- Potential
- Evidence

Why important

Looks like a real hiring product.

Tier 3 Features (Winner Features)

These separate you from most teams.

13. Career Trajectory Score

What it does

Measures career growth.

Candidate A:

```
Intern
↓
Developer
↓
Senior Developer
↓
Lead
```

Candidate B:

```
Developer
↓
Developer
↓
Developer
```

A gets higher score.

Why important

Recruiters care about growth.

14. Potential Score

What it does

Predicts ability to grow.

Example:

JD:

```
LLM Engineer
```

Candidate:

```
Python  
Transformers  
Deep Learning
```

Missing LLM experience.

Potential still high.

Why important

Finds future stars.

15. Hidden Gem Detector

What it does

Finds overlooked talent.

Example:

```
Tier 3 college  
No FAANG
```

but

```
Strong projects  
Strong skills  
Strong growth
```

Output:

```
Hidden Gem Candidate
```

Why important

Directly solves the hackathon problem.

16. Risk Analysis

What it does

Shows hiring risk.

Example:

```
Low Risk
Medium Risk
High Risk
```

Based on:

- Missing skills
- Domain mismatch
- Experience gaps

Why important

Recruiters think about risk.

17. Why Candidate A > Candidate B

What it does

Explains ranking differences.

Example:

```
A ranked higher because:

+ Better project relevance
+ Better growth
+ Better domain experience
```

Why important

Very few teams build this.

Tier 4 Features (Memorable Features)

These help win demos.

18. Recruiter Recommendation Engine

What it does

Acts like an AI recruiter.

Output:

```
Interview Immediately
```

```
Reason:
```

```
Strong fit
```

```
High potential
```

```
Low risk
```

Why important

Feels like a recruiter assistant.

19. Anonymous Hiring Mode

What it does

Hides:

```
Name
```

```
College
```

```
Company
```

Shows:

Skills
Projects
Experience

Why important

Fair hiring and AI ethics angle.

20. Hiring Funnel Visualization

What it does

Visualizes filtering.

5000
↓
1200
↓
300
↓
50
↓
10

Why important

Amazing for demos and PPT.

Feature Judges Will Remember Most

If a judge only remembers 5 things from your demo, you want them to be:

1. **Hidden Gem Detector**
2. **Potential Score**
3. **Career Trajectory Score**

4. **Why Candidate A > Candidate B**

5. **Recruiter Recommendation Engine**

Those 5 features transform the project from a resume matcher into an **AI Recruiter Copilot**, which is the strongest positioning for this hackathon.